

✓ XXXXXXXXXX Shor's Algorithm

You will create a simulation of Shor's Algorithm without necessarily detailing the qubit hilbert space vectors numerically.

You will also need to use the matplotlib package. You can install it by running `pip install matplotlib`.

Submit this assignment by completing this notebook and submit it.

Optionally, you can write your answers to written answer questions on a separate document if you want to.

```
import numpy as np
import math
import cmath

import matplotlib.pyplot as plt

# some numpy setups
r2 = math.sqrt(2)
np.set_printoptions(precision=4)

# mathematical constants and constructs
e = math.e
pi = math.pi

# compute e ** (i * t)
# which gives a complex number with just a phase (dictated by t)
def ei(t):
    return e ** (1j * t)
```

✓ Part 1: create the n-qubit QFT matrix

1.a) Program a function for creating matrices of the QFT operator. (10 pts)

$$QFT_M = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{M-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(M-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{M-1} & \omega^{2(M-1)} & \dots & \omega^{(M-1)(M-1)} \end{bmatrix}$$
$$\omega = e^{2\pi i/M}$$

Then, test this function on a qubit state vector given to you.

Think of it as a state of the qubits after measuring the lower wire in the Shor's algorithm

```
def qft_matrix(n):
    N = 2 ** n # Dimension of the QFT matrix
    omega = np.exp(2j * np.pi / N) # Root of unity
    QFT = np.array([[omega**(i * j) for j in range(N)] for i in range(N)]) / np.sqrt(N)
    return QFT
```

After you coded your QFT matrix generation function, run the following blocks:

```
# A sample state is created for you here
m_ = 8
some_state = np.zeros(2 ** m_)

# can you guess what this state means? (you do not have to)
rg = range(3, 2 ** m_, 6)
some_state[list(rg)] = 1 / math.sqrt(len(rg))

# this uses the qft_matrix() function you coded
# the QFT operator is applied to the state
after_qft = qft_matrix(m_) @ some_state
# amplitude -> measurement probability
probs = after_qft * np.conjugate(after_qft)
```

You will plot the probabilities of measurement of the resulting state below by using the matplotlib package

1.b) Looking at the graph, how many peaks are there? Thus, what do you think the r resulting from this run of Shor's algorithm is? (15 pts)

You can type your answers right here in the notebook or submit a writeup in a pdf document.

✓ ANSWER

Analyzing the graph, I see that there are 6 peaks visible (including the one located at index 0). In order to calculate the peaks, I must divide the total number of probabilities (which in this case is $M = 2^m = 2^8 = 256$) by the distance between peaks which is called r (which in this case is about 43). So, $256/43 = 5.9$ which is almost equal to 6, the number of peaks detected on the graph. Therefore, the number of peaks depicted on the graph are correct according to periodicity r for when $m=8$.

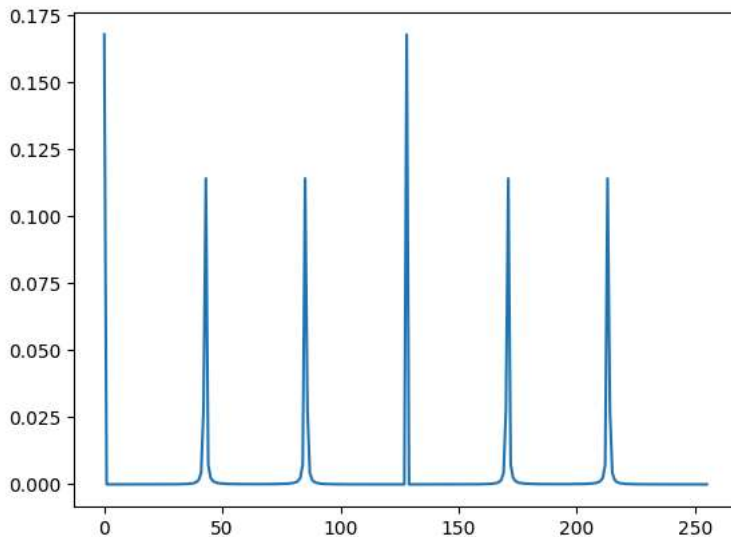
```
# plot the state now in the spectral domain
plt.plot(probs)
# Define an appropriate threshold based on the given data
threshold = 1e-2 # Chosen to capture significant peaks

total_probabilities = len(probs)
print("Total number of probabilities:", total_probabilities)

# Identify indices where the probabilities exceed the threshold
peak_indices = np.where(np.abs(probs) > threshold)[0]

# Extract and print the probabilities at the identified peak indices
peak_probs = probs[peak_indices]
print("Indices of Peaks:", peak_indices)
print("Probabilities of Peaks:", peak_probs)
```

↪ Total number of probabilities: 256
Indices of Peaks: [0 42 43 85 86 128 170 171 213 214]
Probabilities of Peaks: [0.168 +0.0000e+00j 0.0278-1.7284e-20j 0.1142-7.2866e-19j
0.1142+2.9623e-18j 0.0278-2.2366e-19j 0.168 +6.6013e-32j
0.0278-4.0506e-19j 0.1142-4.9411e-19j 0.1142-3.1748e-18j
0.0278+1.6321e-19j]



✓ Part 2: Suitability of the modulo and abstractly simulating the shor's algorithm

Here, you will complete 2 code TODOs and use the function you coded to assist you in answering the given questions.

```
...
creates the list of modulus used in Shor's algorithm
a ** r % N for r in 0, 1, 2, ..., until (= 1 mod N) is reached again
...

def shor_modulo_sim(N, a):
    # start with first element of the modulo 'wave'
    # which is a ** r % N [r = 0] = 1
    res = [1,]
    while True:
        mnext = (res[-1] * a) % N
```

```

        if mnext == 1:
            break
        if mnext in res:
            break
        res.append(mnext)
    return res

print(shor_modulo_sim(57, 8))
print(shor_modulo_sim(45, 7))
print(shor_modulo_sim(63, 16))

```


 [1, 8, 7, 56, 49, 50]
 [1, 7, 4, 28, 16, 22, 19, 43, 31, 37, 34, 13]
 [1, 16, 4]

2.a) Code the function that checks if a certain a is suitable for factoring a certain N. (16 pts)

Think about what conditions you need to check. Recall them from your book/notes/memory.

```

from math import gcd

'''
Check the suitability of a given N and a for Shor's algorithm
returns a tuple of (bool, str)
the boolean denotes if it is suitable
the string denotes the reason why (if suitable == False)
'''

def shor_modulo_suitability(N, a):
    # TODO: create code that checks the suitability of the given N and a
    # hint: there are 3 conditions you should check, what are they?
    # hint: utilize the shor_modulo_sim() function given above

    # Condition 1: a must be coprime to N
    if gcd(a, N) != 1:
        return (False, f"Base {a} is not coprime to {N} (gcd = {gcd(a, N)}).")

    # Condition 2: a^2 % N must not equal 1
    if pow(a, 2, N) == 1:
        return (False, f"Base {a} is unsuitable because a^2 ≡ 1 (mod {N}).")

    # Condition 3: a^r % N equal N for some r
    r = 1
    while r < N:
        if pow(a, r, N) == 1:
            break
        r += 1
    if r == N:
        return (False, f"Base {a} does not have a valid order r modulo {N}.")

    # All conditions are met
    return (True, "All conditions are met and r is " + str(r) + ".")

```

2.b) Answer the following question about suitability of running Shor's algorithm: (24 pts)

- Is $a = 8$ suitable for factoring $N = 57$? Explain why or why not.
- Is $a = 7$ suitable for factoring $N = 45$? Explain why or why not.
- Is $a = 16$ suitable for factoring $N = 63$? Explain why or why not.

You can type your answers right here in the notebook or submit a writeup in a pdf document.

✓ ANSWER

In order to figure out if a is suitable for factoring N , I must first check the greatest common demoninator of a and N , must be coprime (only be 1). Next I must check if $(a^2 \bmod N)$ is not equal to 1. Finally I must find the order r where r will satisfy the following; $a^r \bmod N = 1$.

- When $a = 8$ and $N = 57$
 - $\gcd(8, 57) = 1$; hence co-prime
 - $8^2 \bmod 57 = 64 \bmod 57 = 7$, not equal to 1 hence fine
 - $8^r \bmod 57 = 1$ when $r = 6$, thus valid order exists
 - Therefore 8 is suitable for factoring 57
- When $a = 7$ and $N = 45$

- $\gcd(7, 45) == 1$; hence co-prime
- $7^2 \bmod 45 = 49 \bmod 45 = 4$, not equal to 1 hence fine
- $7^r \bmod 45 = 1$ when $r = 12$, thus valid order exists
- Therefore 7 is suitable for factoring 45

3. When $a = 16$ and $N = 63$

- $\gcd(16, 63) == 1$; hence co-prime
- $16^2 \bmod 63 = 256 \bmod 63 = 4$, not equal to 1 hence fine
- $16^r \bmod 63 = 1$ when $r = 3$, thus valid order exists
- Therefore 16 is suitable for factoring 63

Utilize the function you coded here to answer question 2b!

```
print(shor_modulo_suitability(57, 8))
print(shor_modulo_suitability(45, 7))
print(shor_modulo_suitability(63, 16))
print(shor_modulo_suitability(51, 4))
print(shor_modulo_suitability(57, 10))
```

```
(True, 'All conditions are met and r is 6.')
(True, 'All conditions are met and r is 12.')
(True, 'All conditions are met and r is 3.')
(True, 'All conditions are met and r is 4.')
(True, 'All conditions are met and r is 18.')
```

3.a) Create a function to simulate the Shor's algorithm (15 pts)

Instead of laying out the qubits' Hilbert vectors in full, we just need to create an abstract simulation.

Given a number that the lower wire collapse to, it should output a list of periodicity elements in the Hilbert vector before QFT, and the peaks of the state after QFT.

```
...
simulates the shor algorithm in a more abstract domain without fully laying out Hilbert vectors
N is the number we want to factor
a is the base of the modulo exponential we choose
M is the size of the upper wire, should be a power of 2
lower_collapse is the number in the lower wire we assume collapse to
...
def shor_sim(N, a, M, lower_collapse):
    # TODO: code an abstract simulation of Shor's algorithm
    # hint 1: start by checking the suitability of running Shor's algorithm with this N and a
    suitability, reason = shor_modulo_suitability(N, a)
    if not suitability:
        raise ValueError(f"Shor algo not suitable for N={N}, a={a}: {reason}")

    # get the modulo sequence
    mods = shor_modulo_sim(N, a)
    # hint 2: how does r relate to the modulo sequence?
    # also, note that lower_collapse should be a number within mods
    # you can get its index within the list using mods.index(lower_collapse)
    # how can you utilize this?
    if lower_collapse not in mods:
        raise ValueError(f"Lower collapse value {lower_collapse} not in modulo sequence {mods}.")
    c_index = mods.index(lower_collapse)
    r = len(mods)

    # create the periodicity state as a list
    # hint 3: it looks like this [c, c+r, c+2*r, ...]
    periodicity_state = [c_index + i * r for i in range(M // r)]

    # and create the list for the peak of the state after QFT
    # hint 4: there are M elements in the Hilbert vector of the upper wire qubits
    # and there are r peaks. What does it look like?
    # use np.round() to round a number to nearest integer if needed

    qft_peaks = [np.round(i * M / r) % M for i in range(r)]

    return periodicity_state, qft_peaks
```

3.b) Simulate Shor's algorithm using the function you just coded. (20 pts)

i. With $N = 51$, $a = 4$, $M = 4096$

and the lower wire collapses to 13.

What are the first few elements in the periodicity state list?

What are the peaks of the state after QFT?

ii. With $N = 57$, $a = 10$, $M = 4096$

and the lower wire collapses to 34.

What are the first few elements in the periodicity state list?

What are the peaks of the state after QFT?

You can type your answers right here in the notebook or submit a writeup in a pdf document. Feel free to just use/copy paste your code output.

```
# Utilize the function you coded here to answer question 3b

# periodicity_state, qft_peaks = shor_sim(...)
# Case i: N = 51, a = 4, M = 4096, lower_collapse = 13
N1, a1, M1, lower_collapse1 = 51, 4, 4096, 13
periodicity_state1, qft_peaks1 = shor_sim(N1, a1, M1, lower_collapse1)

# Case ii: N = 57, a = 10, M = 4096, lower_collapse = 34
N2, a2, M2, lower_collapse2 = 57, 10, 4096, 34
periodicity_state2, qft_peaks2 = shor_sim(N2, a2, M2, lower_collapse2)

# Results for both cases
{
  "Case i": {
    "Periodicity State (first few elements)": periodicity_state1[:10],
    "QFT Peaks": qft_peaks1
  },
  "Case ii": {
    "Periodicity State (first few elements)": periodicity_state2[:10],
    "QFT Peaks": qft_peaks2
  }
}

{ 'Case i': { 'Periodicity State (first few elements)': [3,
7,
11,
15,
19,
23,
27,
31,
35,
39],
'QFT Peaks': [0.0, 1024.0, 2048.0, 3072.0]},
'Case ii': { 'Periodicity State (first few elements)': [7,
25,
43,
61,
79,
97,
115,
133,
151,
169],
'QFT Peaks': [0.0,
228.0,
455.0,
683.0,
910.0,
1138.0,
1365.0,
1593.0,
1820.0,
2048.0,
2276.0,
2503.0,
2731.0,
2958.0,
3186.0,
3413.0,
```

3641.0,
3868.0]]}